

Лекция 8. Грамматики. Машины Тьюринга.

1 Полезный частный случай

Пусть $L \subseteq \{a\}^*$, то есть L — язык в однобуквенном алфавите. Тогда L — регулярный $\iff$ он представим в виде объединения конечного числа арифметических прогрессий, где под арифметической прогрессией понимается регулярный язык вида

$$L_{nm} = \{a^n \cdot (a^m)^t, t \in \mathbb{N}_0\}$$

Доказательство. ($\Leftarrow$): $L_{nm} = a^n \cdot (a^m)^*$ — регулярное выражение, а в силу того, что таких арифметических прогрессий конечное число, мы можем пронумеровать их и составить из них регулярное выражение:

$$L = L_{n_1 m_1} + L_{n_2 m_2} + \dots + L_{n_k m_k} \text{ — регулярное выражение}$$

Следовательно, язык L регулярный.

($\Rightarrow$): пусть L — регулярный язык, тогда по нему можно построить ПДКА. Возьмем произвольное слово из языка, пусть ПДКА принимает это слово в некотором финальном состоянии q_f . Заметим, что q_f либо лежит в каком-нибудь цикле, либо не лежит.

Пусть в ПДКА $|Q| + 1$ состояний, тогда возможные простые циклы имеют длину не больше $|Q|$. Теперь рассмотрим кратчайшие пути к финальным состояниям из начального (то есть те пути, которые не посещают одну и ту же вершину два раза): таких путей также будет не более $|Q| + 1$. В множестве арифметических прогрессий $L_{nm} : m \leq |Q|$ содержатся все арифметические прогрессии, в объединении которых лежит язык L , так как каждое слово из языка лежит в некоторой арифметической прогрессии, которая, в свою очередь, целиком лежит в языке L (это следует из леммы о разрастании). Покажем, что подходящих нам арифметических прогрессий, на самом деле, конечное число: зафиксируем $m \leq |Q|$, рассмотрим следующие арифметические прогрессии:

$$a^{n_1}(a^m)^t, a^{n_2}(a^m)^t, \dots, a^{n_k}(a^m)^t$$

Заметим, что как только $k > m$, среди прогрессий найдутся хотя бы $k - m$ вложенных (так как существует всего m остатков от деления на m , и при $k > m$, по принципу Дирихле, хотя бы $k - m$ арифметических прогрессий будут иметь неуникальные остатки), следовательно, есть не более m арифметических прогрессий. Отсюда легко получаем, что число арифметических прогрессий, в объединении которых лежит язык L , не превосходит $(|Q| + 1) \times (|Q| + 1)$, то есть, это число конечно. $\square$

2 Грамматики

Определение 2.1. Грамматика G — это набор из четырех множеств (N, T, P, S) , где

- N — алфавит нетерминальных символов
- T — алфавит терминальных символов (символы из алфавита формального языка), $T \cap N = \emptyset$
- P — конечное множества правил вида $\alpha \rightarrow \beta$, где $\alpha \in (N \cup T)^* N (N \cup T)^*$, $\beta \in (N \cup T)^*$
- $S \in N$ — начальный знак (аксиома) грамматики

Пример 2.1. Зададим с помощью грамматики наш любимый язык $a^n b^n$:

$$P = \{S \rightarrow \varepsilon, S \rightarrow ab, S \rightarrow aSb\}, \quad N = \{S\}, \quad T = \{a, b\}$$

Если последовательно применять правила, пока не получится цепочка, состоящая только из терминальных символов, то получатся только слова вида $a^n b^n$.

$$S \rightarrow aSb \rightarrow aaSbb \rightarrow aaabbb = a^3 b^3$$

Замечание 2.1. Грамматика задает язык L , если любые слова из L можно вывести из S , применяя правила из P и только их.

Пример 2.2. Построим грамматику по ДКА:

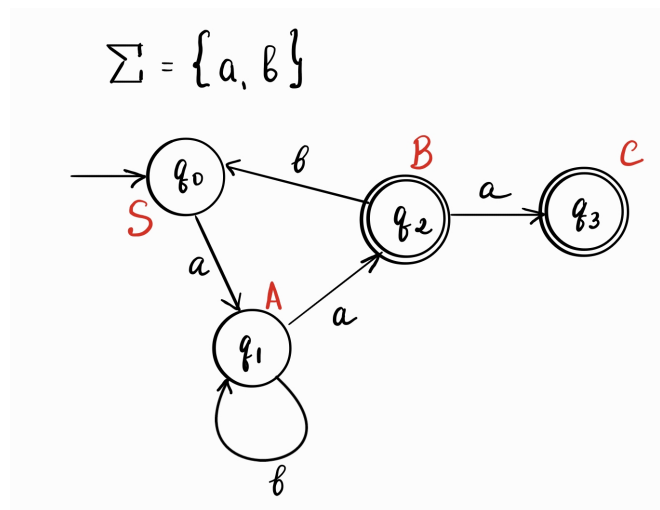


Рис. 1: вот по такому ДКА

Поставим в соответствие каждому состоянию некоторый нетерминал, и построим множество правил:

$$P = \{S \rightarrow aA, A \rightarrow bA, A \rightarrow aB, B \rightarrow bS, B \rightarrow aC, B \rightarrow \varepsilon, C \rightarrow \varepsilon\}$$

Например, слово "aa" мы можем получить следующим образом: $S \rightarrow aA \rightarrow aaB \rightarrow aa$.

Определение 2.2. Типы грамматик (иерархия Хомского):

- Грамматику, для которой нет дополнительных ограничений, называют грамматикой **без ограничений** (типа 0). (то есть в правилах нетерминалы и терминалы могут стоять как угодно, а не только один нетерминал слева, как в рассматриваемых нами грамматиках)

Практического применения не имеют, так как слишком сложная структура. Распознается МТ.

- Если каждое правило грамматики кроме $S \rightarrow \varepsilon$ имеет вид $\alpha \rightarrow \beta$, где $|\alpha| \leq |\beta|$, и в случае, если правило $S \rightarrow \varepsilon \in P$, то символ S не встречается в правых частях правил, то грамматику называют **контекстно-зависимой** или **неукорачивающей** (типа 1)

Используются для анализа естественных языков.

- Если каждое правило грамматики имеет вид $A \rightarrow \beta$, где $A \in N$, $\beta \in (N \cup T)^*$, то её называют **контекстно-свободной** (типа 2)

Широко применяются для описания синтаксиса компьютерных языков. Распознаются при помощи конечного автомата со стеком.

- Если каждое правило грамматики имеет вид либо $A \rightarrow xB$, либо $A \rightarrow x$, где $A, B \in N$, $x \in T^*$, то грамматику называют **регулярной** (типа 3) Совпадает с множеством регулярных. Используется для ассемблера, командных процессов и др.

3 Машины Тьюринга

Определение 3.1. Машина Тьюринга (МТ) — это набор $(A, Q, Q_f, \delta, q_0, \Lambda)$, где

- A — конечное множество, называемое алфавитом MT
- Q — конечное множество, называемое множеством состояний MT : $A \cap Q = \emptyset$
- Q_f — множество финальных состояний MT : $Q_f \subseteq Q$
- δ — функция (таблица) переходов: $\delta : A \times Q \rightarrow A \times \{-1, 0, 1\} \times Q$
- q_0 — начальное состояние: $q_0 \in Q$
- Λ — пустой символ: $\Lambda \in A$

Замечание 3.1. Будем считать, что задача является алгоритмически разрешимой $\iff$ по ней можно построить машину Тьюринга.

Замечание 3.2. Будем считать, что машина Тьюринга завершает работу, как только она попадает в финальное состояние. При этом, если машина Тьюринга попала в финальное состояние, это не значит, что она приняла слово.

Работа машины Тьюринга выглядит следующим образом: у нас имеется лента, она же память (бесконечная), разделенная на ячейки. В самом начале управляющее устройство машины Тьюринга смотрит на какую-то одну ячейку, при этом устройство находится в состоянии q_0 . На вход MT подается слово $x \in A^*$. Например, $A = \{a, b, \Lambda\}$, $x = aba$ (считаем, что в самом начале управляющее устройство указывает на первый символ поданного на вход слова). Мы работаем с детерминированными машинами Тьюринга, то есть, по состоянию и символу мы можем однозначно определить переход.

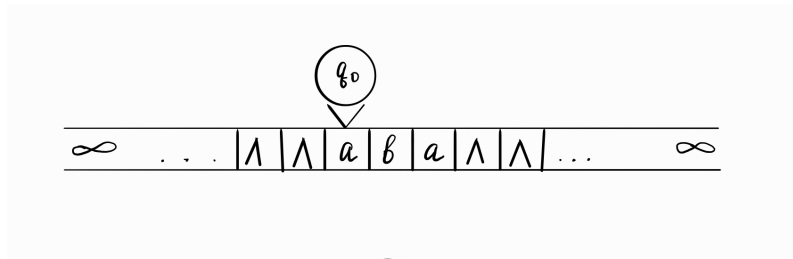


Рис. 2: начальное состояние MT

Пусть $\delta(a, q_0) = b \times \{+1\} \times q_1$. Это значит, что символ a поменяется на b , управляющее устройство сдвинется вправо и перейдет в состояние q_1 .

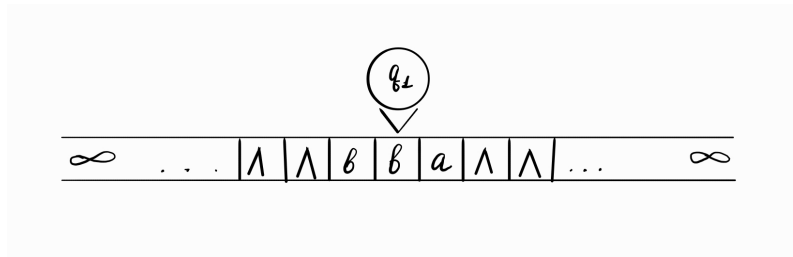


Рис. 3: после перехода

Множество всех машин Тьюринга счётно. (Любую машину Тьюринга можно представить конечным словом из 0 и 1, определив способ кодирования состояний и символов алфавита).

Пример 3.1. Построим машину Тьюринга и закодируем ее: пусть

$$A = \{a, b, c, \Lambda\}, \quad Q = \{q_0, q_1\}, \quad Q_f = \{q_1\}$$

$$\delta : \begin{array}{c|ccccc} & a & b & c & \Lambda \\ \hline q_0 & a, +1, q_1 & b, +1, q_1 & b, -1, q_1 & \Lambda, 0, q_1 \\ \hline q_1 & & & & \end{array}$$

Будем кодировать каждое слово четырьмя битами, чтобы обозначить это, запишем 4 единицы (до первого нуля). 0 — разделяющий символ, его пропускаем. Теперь закодируем алфавит: Λ — 0000, a — 0001, b — 0010, c — 0011. Для удобства символы алфавита будут начинаться с 0, а символы состояний — с 1. Договоримся, что 0 после множества состояний отделяет Q от Q_f , также 0 будет отделять Q_f от переходов.

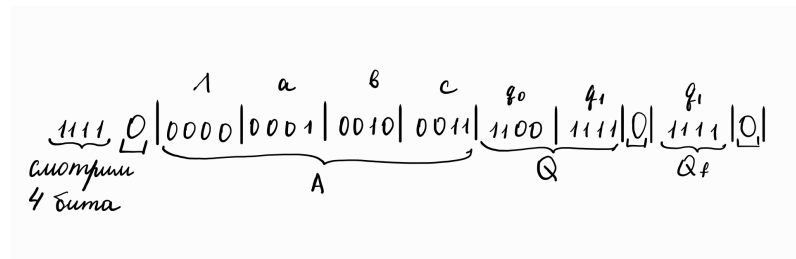


Рис. 4: вот так закодируем

Перечислять переходы будем следующим образом: первые 4 бита — символ алфавита, следующие 4 бита — состояние, далее 4 бита — новый символ алфавита, потом 2 бита (тут всегда 2 бита) на $\pm 1, 0$, и 4 бита на новое состояние. (Почти везде присутствует магическое число 4, потому что мы изначально выбрали именно 4 бита для кодировки всех слов).

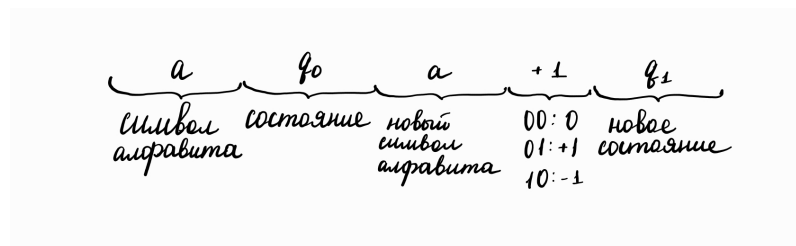


Рис. 5: вот так закодируем переход

Ещё для удобства любое количество нулей в начале будем отбрасывать.

Таким образом, любую машину Тьюринга мы можем закодировать конечным словом из 0 и 1, а значит, всего машин Тьюринга счётное количество.

Замечание 3.3. Так как языков несчётное количество, значит, не все языки алгоритмически разрешимы.

Замечание 3.4. Другие формализации МТ (в этом семестре эти МТ эквивалентны, то есть задают одно и то же множество алгоритмически разрешимых задач):

- k -ленточная МТ

- *MT с лентой, бесконечной в одну сторону*
- *MT с лентой, бесконечной в обе стороны*
- *детерминированная / недетерминированная MT*